Applied AI System

Taanyaa Haridass Prasad

CodePath AI110

Table of Contents

01

Original System

Slide 04

02

AI Feature

Slide 05

03

Demo

Slide 07

04

Reflection

Slide 16

Game Glitch Investigator Goals

01

Goal 1
Debugging

Manually test the app, identify bugs, and use AI tools such as Claude Code and ChatGPT to implement fixes and tests.

02

Goal 2
Testing System (Reliability)

Developed a structured testing harness and generate a summary report.

03

Goal 3
AI Evaluation (Gemini)

Integrated Gemini API to analyze results to evaluate the system.

Original System

01

Interactive Guessing Game

User selects a difficulty level, tries to guess a secret number, and receives feedback like too high or too low.

02

Functions

Three main functions which include input parsing, guess checking, and score updating all of which were organized in a logic_utils.py file.

03

Overall System

System was fully functional as a game but there was no structured testing or evaluation.

Enhanced System

01

Test Harness

Runs predefined inputs against core functions.

Checks whether functions behave as expected.

Compares actual output with expected output and records pass/fail.

Results summarized into a report.

02

Gemini API

Summary report sent to Gemini AI model to analyze results and provide reliability evaluation.

Returns structured response that indicates whether results are consistent and if any failures occur.

Provides reliability score and offers suggestions for improvement.

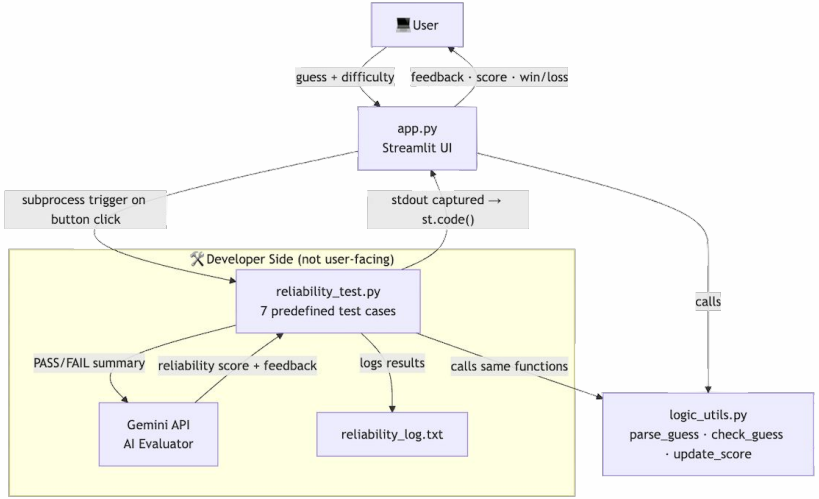
03

Logs + Guardrails

Tests results and any errors that occur are logged in a separate file.

Error handling is implemented as a guardrail mechanism to avoid code crashing.

System Architecture



Demo

Deploy

Settings

Difficulty

Normal

Range: 1 to 50

Attempts allowed: 6

🎮 Game Glitch Investigator

An AI-generated guessing game.

Make a guess

Guess a number between 1 and 50. Attempts left: 6

Developer Debug Info

Enter your guess:

Submit Guess 🚀

New Game 🔄

Show hint

Built by an AI that claims this code is production-ready.

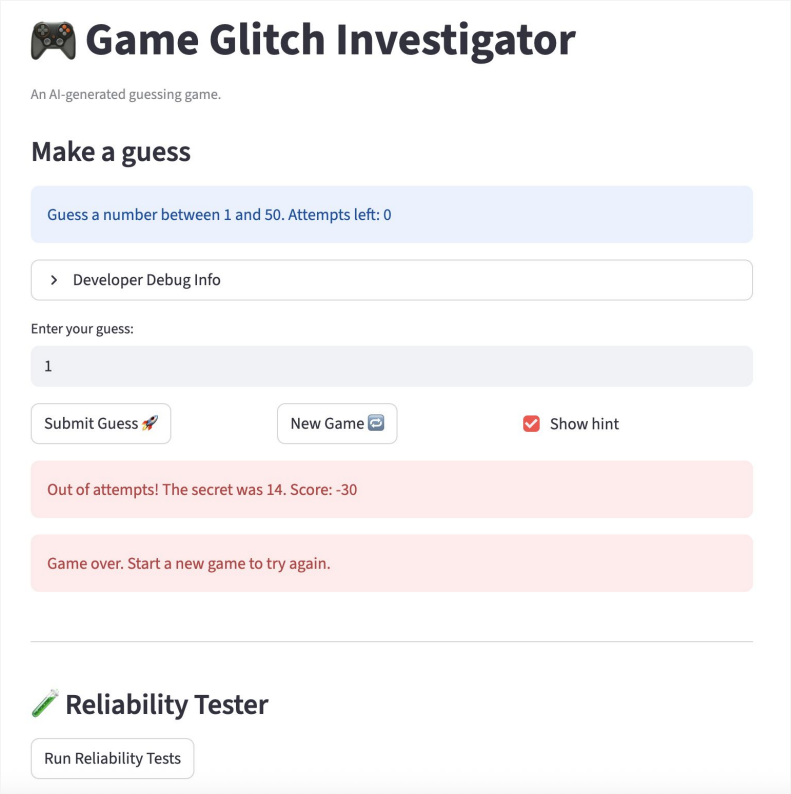
🧪 Reliability Tester

Run Reliability Tests

Case 1: Game Lost

User did not guess the secret number within the given number of attempts.

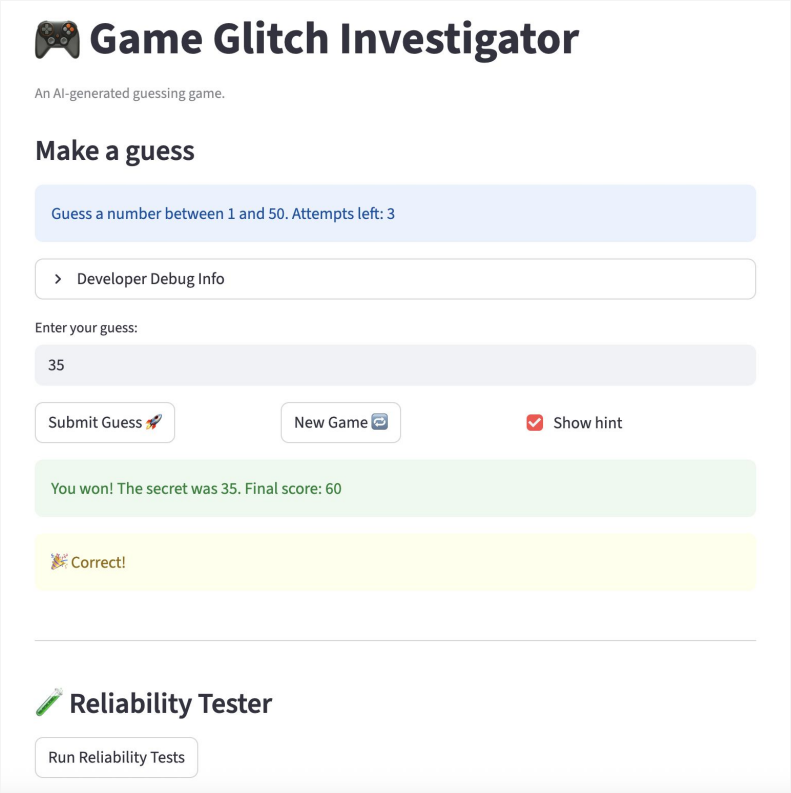
Game displays the secret number, the score, and prompts user to start a new game.



Case 2: Game Won

User did guess the secret number within the given number of attempts.

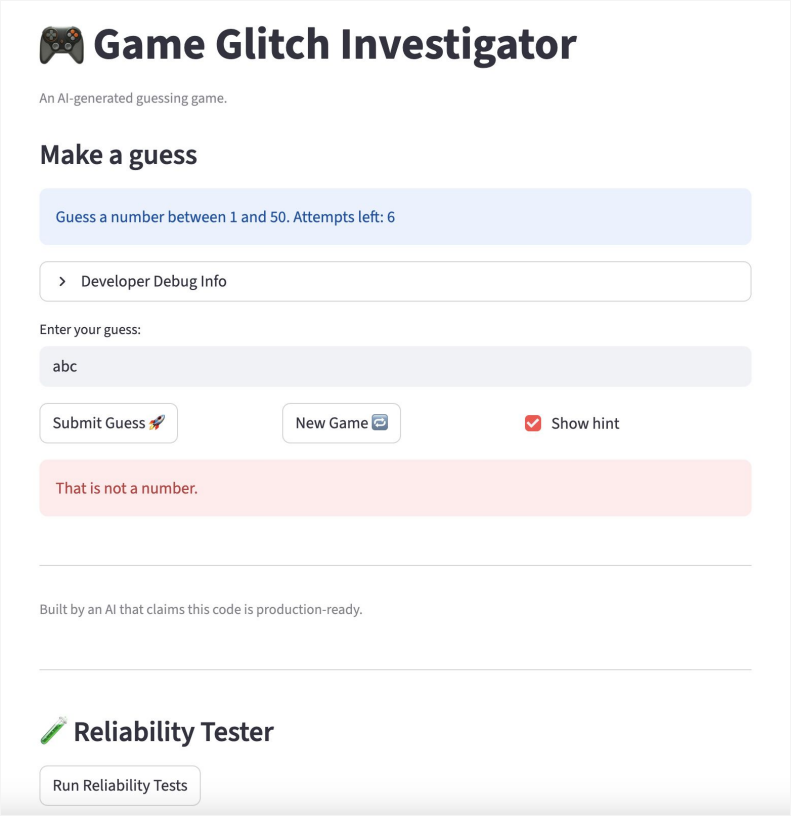
Game displays the secret number and the score. Game also includes a balloon animation if number was guessed.



Case 3: Invalid Input

User enters an input that cannot be converted to an integer.

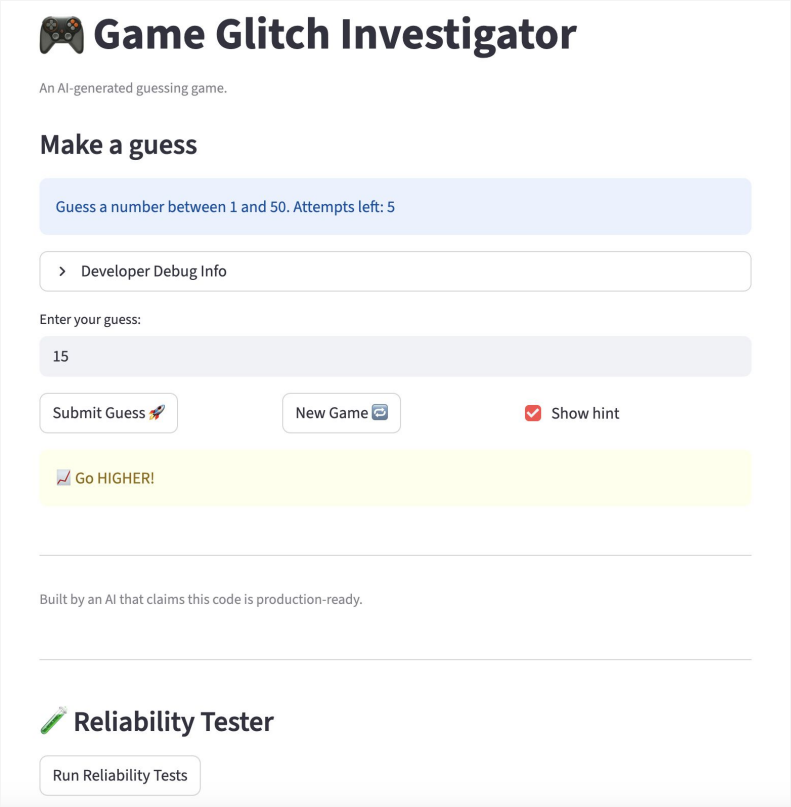
Game displays a message indicating the user's input is not a number.



Case 4: Too High

User enters a number that was higher than the secret number.

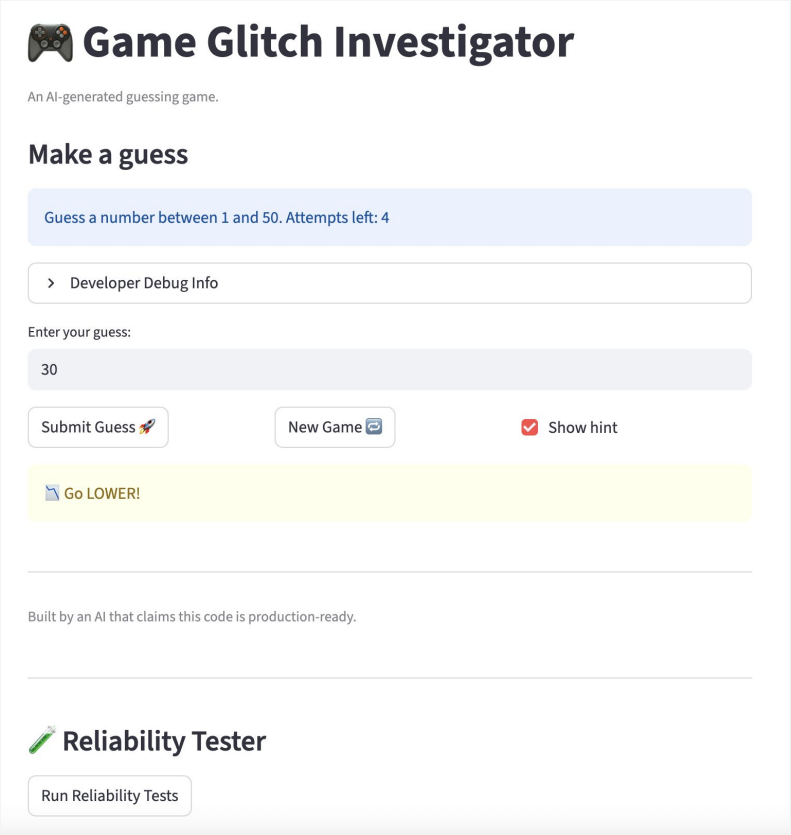
Game displays a hint message indicating the user's input is higher than the secret number.



Case 5: Too Low

User enters a number that was lower than the secret number.

Game displays a hint message indicating the user's input is lower than the secret number.



Reliability Testing

Gemini API returns evaluation report.

🧪 Reliability Tester

Run Reliability Tests

⏳ Sending results to Gemini for evaluation...

===== GEMINI RELIABILITY REPORT ===== Here is the evaluation:

1. **Are the results correct and consistent?** Yes, all results show `expected` matching `got`, indicating that the tests passed successfully as defined.
2. **Are there any concerning failures?** No, all tests passed (`[PASS]`). There are no failures to be concerned about.
3. **Overall reliability score (0.0 to 1.0 as a percentage):** 1.0 (100%) - Based solely on the provided test results, the game logic passes all specified checks.
4. **Short recommendation for improvement:** Expand test coverage to include boundary conditions for guesses (e.g., min/max numbers), score deductions for "Too High" guesses, scenarios with multiple wrong attempts, and edge cases for input parsing (e.g., empty string, whitespace).

===== TEST SUMMARY ===== [PASS] Correct guess should be a win [PASS] Guess too high should return Too High [PASS] Guess too low should return Too Low [PASS] Valid number string should parse correctly [PASS] Non-number guess should fail to parse [PASS] Win on first attempt should score 90 [PASS] Wrong guess Too Low should deduct 5 points

7/7 tests passed

Reliability Testing

Example of Gemini's evaluation report when bugs are present.

🧪 Reliability Tester

Run Reliability Tests

⏳ Sending results to Gemini for evaluation...

===== GEMINI RELIABILITY REPORT ===== Here's an evaluation of the test results:

1. **Are the results correct and consistent?** The results are inconsistent regarding core game logic. The `Correct guess should be a win` test fails by returning "Too Low", while the `Guess too high should return Too High` test fails by returning "Win". This suggests a fundamental and contradictory misunderstanding of the win condition and the "Too High" condition within the game logic. Other results (parsing, "Too Low" feedback, scoring) are consistent with expectations.
2. **Are there any concerning failures?** Yes, the two failures are critically concerning.
   - `Correct guess should be a win` failing means the game cannot be won correctly.
   - `Guess too high should return Too High` failing by returning "Win" means the game can be won incorrectly. These issues render the core gameplay broken and unplayable.
3. **Overall reliability score from 0.0 to 1.0 as a percentage.** 5 out of 7 tests passed. Score: **0.71 (71%)**
4. **Give a short recommendation for improvement.** Prioritize fixing the core game logic for determining `Win`, `Too High`, and `Too Low` conditions. The current implementation is fundamentally flawed and contradictory.

===== TEST SUMMARY ===== [FAIL] Correct guess should be a win [FAIL] Guess too high should return Too High [PASS] Guess too low should return Too Low [PASS] Valid number string should parse correctly [PASS] Non-number guess should fail to parse [PASS] Win on first attempt should score 90 [PASS] Wrong guess Too Low should deduct 5 points

5/7 tests passed

Reflection

One thing I learned from this project is the importance of verifying system behavior rather than assuming correctness.

This project helped me understand how automated testing and AI-based evaluation can complement each other in building more reliable systems.

GitHub Repo Link

https://github.com/taanyaaharidassprasad06/applied-ai-system-project

Thank you